

Experiment 2 Addendum

Line Coding as Waveform Design, Dynamical Systems, and Sequence Detection

Contents

1	Experiment 2: A Broader View of Line Coding	2
2	From Bits to Waveforms	3
3	Waveform Design and Vector Spaces	3
3.1	Pulse Shape Is Also a Design Choice	4
4	Line Coding as a Simplification of Signal-Space Generation	5
5	Memoryless Symbol-by-Symbol Line Coding	5
6	But AMI Is Different	5
7	AMI as a Nonlinear Dynamical System	6
7.1	Why Is This Interesting?	7
8	The AMI State Diagram	7
9	From State Diagram to Trellis	8
10	The Receiver: From Waveform Back to Symbols	8
11	Correlation: The Fundamental Receiver Operation	9
12	Why Matched Filtering Followed by Sampling Implements Correlation	9
13	Memoryless Detection Versus Sequence Detection	10
14	AMI Trellis Decoding	10
15	Why the Trellis Is More Than a Picture	11
16	Bellman's Principle Appears Naturally	11
17	The Viterbi Idea	12
18	A Connection to Optimal Control	12
19	Bellman Equation and the HJB Analogy	13
20	AMI: Complete Transmitter–Receiver Picture	14

21 A Broader View: Memoryless and Stateful Coding	14
22 MLT-3: Another Dynamical System	14
23 AMI and MLT-3 as Finite-State Machines	15
24 Why the Trellis Becomes Useful at the Receiver	16
25 The Geometry Behind the Detection Problem	16
26 A Useful Mental Picture	17
27 Questions for Exploration	18
28 Looking Ahead	18
29 Suggested Extension: Draw the AMI Trellis	19
30 Suggested Extension: Compare AMI and MLT-3	20
31 Final Conceptual Summary	20

1 Experiment 2: A Broader View of Line Coding

The hardware experiment introduces several familiar line coding schemes:

- Unipolar Return-to-Zero (RZ),
- Manchester coding,
- Alternate Mark Inversion (AMI),
- and MLT-3.

At first sight these appear to be different digital circuits implementing different rules for converting bits into voltage waveforms.

There is, however, a deeper unifying viewpoint.

The central question of a communication system is not merely

How do we convert a sequence of bits into a voltage waveform?

but rather

How should information be represented by waveforms so that it can be efficiently generated, transmitted, and subsequently recovered from a noisy observation?

This viewpoint allows us to connect line coding to several ideas already encountered in previous courses and experiments:

- vector spaces and signal representations,
- correlation and inner products,
- matched filtering,
- discrete-time dynamical systems,

- state-space representations,
- trellis diagrams,
- sequence detection,
- dynamic programming,
- the Bellman principle,
- and, ultimately, Hamilton–Jacobi–Bellman (HJB) ideas from optimal and nonlinear control.

The purpose of this addendum is to develop this conceptual chain.

2 From Bits to Waveforms

Let the transmitted binary data sequence be

$$\{b_k\}, \quad b_k \in \{0, 1\}.$$

A line coder converts this sequence into a continuous-time waveform $x(t)$. A very general representation is

$$x(t) = \sum_k a_k p_k(t),$$

where

- a_k is a coefficient or symbol associated with the k th data symbol,
- $p_k(t)$ is the waveform associated with that symbol.

For the simplest linear modulation or line coding systems,

$$p_k(t) = p(t - kT),$$

so that

$$x(t) = \sum_k a_k p(t - kT)$$

where T is the bit interval.

This equation is worth pausing over.

It says that a transmitted waveform is not created from nothing. It is synthesized by taking a collection of elementary waveforms and weighting and shifting them according to the information sequence.

3 Waveform Design and Vector Spaces

The expression

$$x(t) = \sum_k a_k p(t - kT)$$

has a direct interpretation from linear algebra.

The shifted pulses

$$p(t), \quad p(t - T), \quad p(t - 2T), \dots$$

may be regarded as elements of a signal space.

The transmitted signal belongs to their span:

$$x(t) \in \text{span} \{p(t - kT)\}.$$

Thus waveform generation can be viewed as a problem in vector spaces.

The communication system chooses a set of elementary signal vectors and then represents information by selecting an appropriate combination of them.

This is closely related to the familiar finite-dimensional representation

$$\mathbf{x} = \sum_i c_i \phi_i$$

from linear algebra.

The important difference is that the “vectors” here are functions of time.

They live in a signal space rather than in an ordinary Euclidean coordinate space.

3.1 Pulse Shape Is Also a Design Choice

It is tempting to think that once the mapping

$$b_k \longrightarrow a_k$$

has been chosen, the transmitted signal is completely determined.

It is not.

Even in a memoryless symbol-by-symbol system, there remains another important design variable:

$$\text{the pulse } p(t).$$

Two systems can use exactly the same symbol alphabet

$$a_k \in \{-1, +1\}$$

and yet produce substantially different transmitted waveforms by choosing different pulses. For example,

$$p(t) = \begin{cases} 1, & 0 \leq t < T, \\ 0, & \text{otherwise,} \end{cases}$$

produces a rectangular NRZ-type waveform.

On the other hand,

$$p(t) = \begin{cases} 1, & 0 \leq t < T/2, \\ 0, & T/2 \leq t < T, \\ 0, & \text{otherwise,} \end{cases}$$

creates a pulse that occupies only part of the symbol interval.

Thus

$$\text{symbol mapping} \neq \text{complete waveform design.}$$

The pulse shape is itself part of the communication-system design.

4 Line Coding as a Simplification of Signal-Space Generation

A useful way to think about practical waveform design is that we would like to choose a convenient set of basis-like waveforms that make the transmitter simple.

Instead of generating an arbitrary waveform for every possible bit sequence, we attempt to construct the transmitted signal from a relatively small and structured family of elementary pulses.

Thus one may think of pulse design as an attempt to

simplify the generation of the spanning set of waveforms.

For example, a rectangular pulse is extremely attractive in digital hardware because it can be generated using switching between voltage levels.

The hardware implementation therefore reflects a signal-space design choice.

This gives a useful conceptual chain:

Signal space $\rightarrow$ pulse family $\rightarrow$ symbol coefficients $\rightarrow$ transmitted waveform.

The simplicity of line coding hardware is therefore not merely an accident of digital electronics. It is closely related to the deliberate choice of a simple waveform family.

5 Memoryless Symbol-by-Symbol Line Coding

Consider first a conventional memoryless line coder.

The mapping has the form

$$b_k \longrightarrow a_k.$$

For example, for polar signaling,

$$b_k = \begin{cases} 0 & \longrightarrow -A, \\ 1 & \longrightarrow +A. \end{cases}$$

The transmitted signal is

$$x(t) = \sum_k a_k p(t - kT).$$

The important feature is that a_k depends only on b_k .

There is no dependence on previous symbols.

Mathematically,

$$a_k = f(b_k).$$

The sequence of coefficients is therefore obtained using a symbol-by-symbol transformation.

6 But AMI Is Different

AMI provides a particularly interesting example because the amplitude assigned to a binary one depends on what happened previously.

In AMI,

$$0 \longrightarrow 0,$$

whereas successive ones alternate in polarity:

$$+A, -A, +A, -A, \dots$$

Therefore, for a binary one,

$$a_k \in \{-A, +A\},$$

but the choice cannot be made using b_k alone.

We require memory.

Introduce a state

$$s_k \in \{-1, +1\},$$

representing the polarity to be assigned to the next transmitted mark.

Then AMI may be written as

$$a_k = \begin{cases} 0, & b_k = 0, \\ As_k, & b_k = 1. \end{cases}$$

The state evolves according to

$$s_{k+1} = \begin{cases} s_k, & b_k = 0, \\ -s_k, & b_k = 1. \end{cases}$$

Thus the system is no longer simply

$$b_k \longrightarrow a_k.$$

Instead,

$$(b_k, s_k) \longrightarrow (a_k, s_{k+1}).$$

This is the first important conceptual transition.

7 AMI as a Nonlinear Dynamical System

The state equation

$$s_{k+1} = \begin{cases} s_k, & b_k = 0, \\ -s_k, & b_k = 1 \end{cases}$$

is a discrete-time dynamical system.

The input is

$$b_k \in \{0, 1\},$$

the state is

$$s_k \in \{-1, +1\},$$

and the output is

$$a_k \in \{-A, 0, +A\}.$$

We may therefore write the AMI encoder abstractly as

$$\boxed{\begin{aligned} s_{k+1} &= f(s_k, b_k), \\ a_k &= g(s_k, b_k). \end{aligned}}$$

The state space consists of only two states:

$$\mathcal{S} = \{-1, +1\}.$$

This is remarkably small.

Yet that tiny amount of memory is enough to make the output sequence depend on the history of the input sequence.

7.1 Why Is This Interesting?

The same binary sequence can produce different amplitude sequences depending upon the initial state.

For example, consider

$$\{b_k\} = \{1, 0, 1, 1, 0, 1\}.$$

If the initial polarity is

$$s_0 = +1,$$

then the output marks begin as

$$+A, 0, -A, +A, 0, -A, \dots$$

whereas choosing

$$s_0 = -1$$

produces the polarity-reversed sequence.

Thus the encoder is not merely a static lookup table.

It is a state machine.

8 The AMI State Diagram

The state-transition diagram provides the first step toward the trellis representation.

There are two states:

$$+1 \quad \text{and} \quad -1.$$

For a zero input,

$$b_k = 0,$$

the state does not change.

For a one input,

$$b_k = 1,$$

the state changes sign.

The transitions are therefore

+1	$\xrightarrow{0/0}$	+1
+1	$\xrightarrow{1/+A}$	-1
-1	$\xrightarrow{0/0}$	-1
-1	$\xrightarrow{1/-A}$	+1

where the notation

$$b_k/a_k$$

indicates the input/output associated with the transition.
The diagram immediately shows that AMI has memory.

9 From State Diagram to Trellis

A state diagram describes all possible transitions without explicitly showing time.

A trellis unfolds the state diagram across time.

At each time k , we have two possible states:

$$s_k = +1 \quad \text{or} \quad s_k = -1.$$

Each state can have two possible outgoing transitions corresponding to

$$b_k = 0 \quad \text{or} \quad b_k = 1.$$

The trellis therefore has the form

k	$k+1$	$k+2$	$k+3$
+1	+1	+1	+1
	-1	-1	-1
-1	-1	-1	-1
	+1	+1	+1

with branches connecting the appropriate states.

Conceptually,

state diagram	$\xrightarrow{\text{unfold in time}}$	trellis.
---------------	---------------------------------------	----------

The trellis is particularly useful at the receiver because it represents all possible transmitted sequences as paths through a finite state graph.

10 The Receiver: From Waveform Back to Symbols

The transmitter produces

$$x(t) = \sum_k a_k p(t - kT).$$

The receiver observes

$$r(t) = x(t) + n(t),$$

where $n(t)$ represents channel noise and other disturbances.

The receiver's task is to infer the original information sequence.
The conceptual receiver can therefore be represented as

$$\boxed{r(t) \longrightarrow \text{correlation} \longrightarrow \text{symbol/state information} \longrightarrow \text{decision} \longrightarrow \{b_k\}.}$$

For a memoryless system, the decision can often be made one symbol at a time.

For a system such as AMI, however, the correct interpretation of one symbol can depend upon the state.

This is where sequence detection becomes natural.

11 Correlation: The Fundamental Receiver Operation

Suppose the receiver wishes to determine whether a particular waveform $p(t - kT)$ is present.

A natural operation is the inner product

$$\langle r, p_k \rangle = \int r(t) p_k(t) dt,$$

where

$$p_k(t) = p(t - kT).$$

The receiver therefore projects the received waveform onto the relevant signal direction.
This is precisely the continuous-time analogue of taking a dot product.

For a symbol interval, the receiver may calculate

$$z_k = \int_{kT}^{(k+1)T} r(t) p(t - kT) dt.$$

The resulting scalar z_k becomes a sufficient statistic for a suitable signal model.
Thus,

$$\boxed{\text{correlation receiver} = \text{inner-product receiver}.}$$

12 Why Matched Filtering Followed by Sampling Implements Correlation

The correlation operation can be implemented directly by an integrator or correlator.

However, there is an equivalent and extremely important implementation using a matched filter.

Let the transmitted pulse be $p(t)$.

The impulse response of the matched filter is proportional to the time-reversed pulse:

$$h(t) = p^*(T - t).$$

The filter output is

$$y(t) = r(t) * h(t).$$

At the sampling instant $t = T$,

$$y(T) = \int r(\tau) h(T - \tau) d\tau.$$

Substituting

$$h(T - \tau) = p^*(\tau),$$

we obtain

$$y(T) = \int r(\tau) p^*(\tau) d\tau.$$

Therefore,

$$\boxed{y(T) = \langle r, p \rangle.}$$

Hence a matched filter followed by sampling is not merely “similar to” a correlation receiver. It is a physical implementation of the mathematical inner product.

This gives the useful correspondence

$$\boxed{\text{inner product} \longleftrightarrow \text{correlation} \longleftrightarrow \text{matched filtering} + \text{sampling}.}$$

13 Memoryless Detection Versus Sequence Detection

For a memoryless line code, the receiver may compute a statistic z_k and make a decision

$$\hat{a}_k = \arg \min_{a \in \mathcal{A}} d(z_k, a),$$

where $\mathcal{A}$ is the signal alphabet.

The decisions can be made independently.

AMI is different.

Suppose the receiver observes a positive mark.

Is this necessarily a transmitted $+A$ symbol?

Not necessarily.

The interpretation depends on the current polarity state.

The receiver must therefore reason about

$$\boxed{\text{which state are we in?}}$$

and

$$\boxed{\text{which state will we enter next?}}$$

This naturally converts the decoding problem from symbol detection into sequence detection.

14 AMI Trellis Decoding

Let the received statistic associated with symbol k be

$$z_k.$$

For AMI, the possible noiseless outputs are

$$0, \quad +A, \quad -A.$$

Suppose a candidate transition produces output a_k .

A simple squared-error branch metric is

$$\boxed{\gamma_k(s_k, b_k) = |z_k - a_k(s_k, b_k)|^2.}$$

The receiver now asks:

Which path through the AMI trellis gives the smallest total mismatch between the observed signal and a physically possible AMI sequence?

For a candidate sequence,

$$\{b_0, b_1, \dots, b_{N-1}\},$$

the total cost is

$$J = \sum_{k=0}^{N-1} \gamma_k(s_k, b_k).$$

The decoding problem becomes

$$\hat{\mathbf{b}} = \arg \min_{\mathbf{b}} \sum_k \gamma_k(s_k, b_k)$$

subject to the AMI state-transition equations.

This is the essence of trellis-based sequence detection.

15 Why the Trellis Is More Than a Picture

The trellis should not be regarded merely as a convenient drawing.

It represents the complete set of physically admissible signal histories.

Each path through the trellis corresponds to

one possible input sequence

and simultaneously to

one possible state trajectory

and therefore to

one possible transmitted waveform.

Thus,

$$\text{path} \longleftrightarrow \text{state trajectory} \longleftrightarrow \text{symbol sequence} \longleftrightarrow \text{waveform}.$$

The receiver observes the waveform and works backwards through this chain.

16 Bellman's Principle Appears Naturally

The brute-force approach would be to examine every possible input sequence.

For N binary symbols, that would require considering

$$2^N$$

possible sequences.

This rapidly becomes impossible.

The crucial observation is that the problem has a special structure.

Suppose the receiver has reached state s_k .

The cost accumulated up to this point can be summarized by a single quantity

$$J_k(s_k),$$

the minimum cost of reaching state s_k after processing the first k observations.

Bellman's principle of optimality tells us that an optimal path must contain optimal sub-paths.

Therefore,

$$J_{k+1}(s_{k+1}) = \min_{s_k, b_k} [J_k(s_k) + \gamma_k(s_k, b_k)]$$

subject to

$$s_{k+1} = f(s_k, b_k).$$

This is the Bellman recursion.

The exponential sequence-search problem has therefore been transformed into a recursive optimization problem over a small state space.

17 The Viterbi Idea

The recursion above is the essential idea behind the Viterbi algorithm.

At every state and time instant, retain only the best surviving path.

If two different histories arrive at the same state,

$$s_k,$$

only the one having the smaller accumulated cost needs to be retained.

The other path can never become better later if all future possibilities depend only on the current state.

This is precisely Bellman's principle of optimality.

The receiver therefore performs

$$\text{branch metric} \rightarrow \text{path metric} \rightarrow \text{survivor selection} \rightarrow \text{backtracking}.$$

The final survivor path gives the estimated data sequence.

18 A Connection to Optimal Control

At this point the connection with dynamical systems and control becomes particularly clear.

An AMI encoder has

$$s_{k+1} = f(s_k, b_k),$$

which has exactly the structure of a controlled discrete-time dynamical system.

The binary symbol b_k acts as a control/input.

The state is s_k .

The observed mismatch contributes a stage cost

$$\gamma_k(s_k, b_k).$$

The receiver is therefore solving

$$\min_{\{b_k\}} \sum_k \gamma_k(s_k, b_k)$$

subject to

$$s_{k+1} = f(s_k, b_k).$$

This is an optimal-control problem.

The difference is mainly one of interpretation:

Communication	Control	Meaning
State	State	Internal memory
Input bit	Control	Choice of transition
Encoder	Plant dynamics	State evolution
Received sample	Observation	Noisy measurement
Branch metric	Stage cost	Local mismatch
Path metric	Value function	Cost-to-go information
Viterbi recursion	Bellman recursion	Dynamic programming

This is a powerful connection because it shows that sequence detection is not an isolated communication-theory trick.

It is an instance of a much broader mathematical idea.

19 Bellman Equation and the HJB Analogy

In continuous-time optimal control, the Hamilton–Jacobi–Bellman equation describes the evolution of the optimal value function.

In schematic form,

$$-\frac{\partial V}{\partial t} = \min_u \left\{ L(x, u) + \nabla V(x, t)^\top f(x, u) \right\}.$$

The discrete-time analogue is the Bellman recursion

$$V_k(s) = \min_u \{ L_k(s, u) + V_{k+1}(f(s, u)) \}.$$

The trellis decoder has exactly this recursive structure.

For sequence detection,

$$V_k(s) = \min_{b_k} \{ \gamma_k(s, b_k) + V_{k+1}(f(s, b_k)) \}.$$

Thus we may view

Viterbi / trellis decoding

as a finite-state, discrete-time dynamic-programming problem.

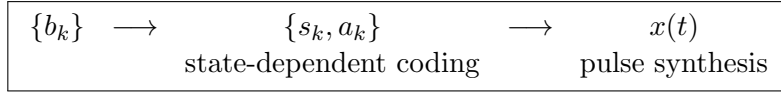
The analogy with HJB is not merely superficial:

HJB $\leftrightarrow$ Bellman recursion $\leftrightarrow$ Viterbi recursion.

The communication problem is therefore another example of the principle that optimal decisions can be constructed recursively from an appropriate value function.

20 AMI: Complete Transmitter–Receiver Picture

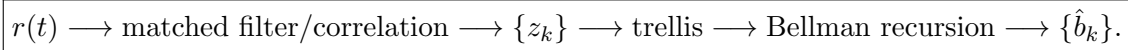
The entire AMI communication system may now be viewed as



followed by the channel

$$x(t) \longrightarrow r(t) = x(t) + n(t),$$

and then



Thus the communication chain becomes



21 A Broader View: Memoryless and Stateful Coding

The distinction between memoryless and state-dependent coding can now be summarized as follows.

Memoryless coding	Stateful coding
$a_k = f(b_k)$	$a_k = g(s_k, b_k)$
No encoder memory	Encoder contains state
Symbol-by-symbol detection often sufficient	Sequence detection may be required
One symbol corresponds to a local decision	A sequence corresponds naturally to a path
Decision regions in signal space	Decision paths in a trellis

The important conceptual point is that introducing memory changes the geometry of the detection problem.

We no longer ask only

Which symbol was transmitted?

We ask

Which admissible sequence of symbols was transmitted?

22 MLT-3: Another Dynamical System

The same viewpoint can be applied to MLT-3.

MLT-3 uses three signal levels,

$$\{-A, 0, +A\}.$$

The output does not depend solely on the current input bit.

Instead, the next output level depends upon the current signal state and the history of previous transitions.

Let the state represent the current output level together with the polarity information required for the next transition.

A convenient state set is

$$\mathcal{S} = \{-A, 0, +A\}.$$

For a zero input,

$$b_k = 0,$$

the state remains unchanged.

For a one input, the state moves through the cyclic sequence

$$0 \rightarrow +A \rightarrow 0 \rightarrow -A \rightarrow 0 \rightarrow +A \rightarrow \dots$$

or the opposite orientation depending upon the chosen initial polarity.

Thus MLT-3 can again be written abstractly as

$$s_{k+1} = f(s_k, b_k)$$

and

$$a_k = g(s_k, b_k).$$

Therefore MLT-3, like AMI, is naturally described as a finite-state dynamical system.

23 AMI and MLT-3 as Finite-State Machines

The two schemes can now be compared from a dynamical-systems viewpoint.

	AMI	MLT-3
Input alphabet	$\{0, 1\}$	$\{0, 1\}$
Output alphabet	$\{-A, 0, +A\}$	$\{-A, 0, +A\}$
Contains memory	Yes	Yes
Finite state representation	Yes	Yes
State-dependent output	Yes	Yes
Trellis representation	Yes	Yes
Sequence detection viewpoint	Yes	Yes
Dynamic programming interpretation	Yes	Yes

The exact state definitions and transition rules differ, but the mathematical architecture is remarkably similar.

This suggests a broader principle:

Whenever a line coder contains memory, its transmitted signal can often be viewed as a path generated by a finite-state dynamical system.

24 Why the Trellis Becomes Useful at the Receiver

Suppose noise causes a received sample to lie somewhere between the ideal levels.

For example,

$$z_k \approx 0$$

might be interpreted as a transmitted zero.

But if the noise is substantial, a single-sample decision may be unreliable.

The trellis gives the receiver additional information.

A candidate symbol is not considered in isolation.

Instead, it must be consistent with

- the previous state,
- the current observation,
- and the state that it produces for the future.

Thus the receiver uses both

local evidence

and

global sequence consistency

.

This is one of the major conceptual advantages of sequence detection.

25 The Geometry Behind the Detection Problem

Recall that the transmitted signal is

$$x(t) = \sum_k a_k p(t - kT).$$

Each possible symbol sequence corresponds to one point in a much larger signal space.

For a memoryless binary system, the possible sequences form a structured collection of points generated by independent symbol choices.

With a finite-state encoder such as AMI or MLT-3, not every arbitrary sequence of output symbols is possible.

The encoder imposes constraints.

Therefore, the valid transmitted waveforms occupy only a subset of the possible signal space.

The trellis is a compact representation of this constrained subset.

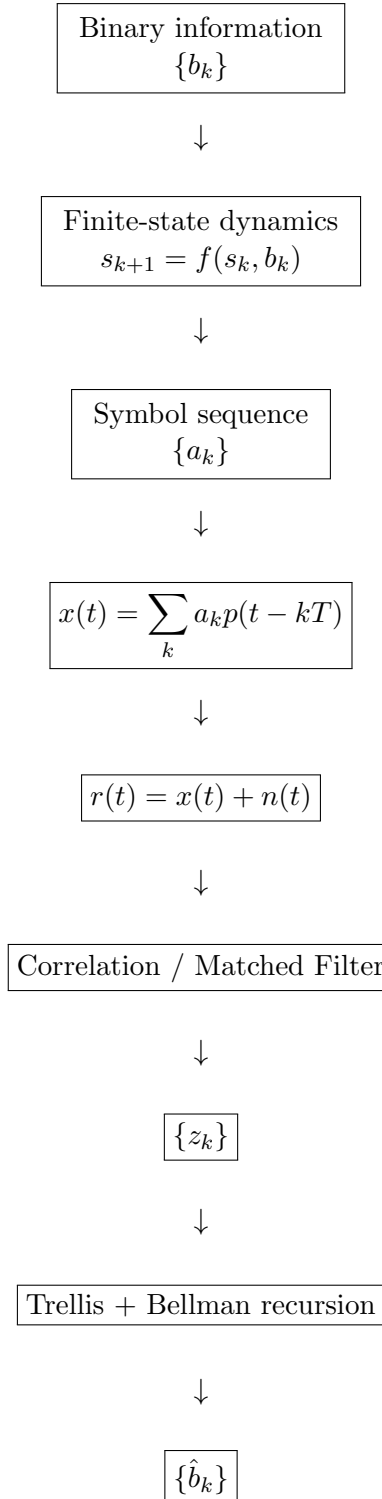
Thus we can think of sequence detection as

find the valid signal-space trajectory closest to the received waveform.

The correlation/matched-filter stage extracts useful coordinates, while the trellis enforces the structural constraints imposed by the encoder.

26 A Useful Mental Picture

Students should try to keep the following picture in mind.



The transmitter therefore moves

information $\rightarrow$ state trajectory $\rightarrow$ waveform,

while the receiver attempts to reverse the process:

waveform $\rightarrow$ state trajectory $\rightarrow$ information.

27 Questions for Exploration

The following questions are intended to encourage students to move beyond circuit-level descriptions.

1. In a memoryless line code, why is the mapping $a_k = f(b_k)$ sufficient?
2. Why does AMI require an internal state?
3. What information does the AMI state s_k actually remember?
4. Can the AMI encoder be represented as a state-space dynamical system?
5. What changes when the state diagram is unfolded into a trellis?
6. Why does each valid transmitted sequence correspond to a path through the trellis?
7. Why can a receiver not necessarily decode a state-dependent line code using independent symbol decisions?
8. How does correlation convert the received waveform into a scalar decision statistic?
9. Why is a matched filter followed by sampling equivalent to computing an inner product?
10. What is the physical meaning of a branch metric in a trellis decoder?
11. Why does the total path metric naturally become a sum of branch metrics?
12. How does Bellman's principle reduce the complexity of sequence detection?
13. In what sense is Viterbi decoding a dynamic-programming algorithm?
14. What is the analogy between the Bellman recursion and the Hamilton–Jacobi–Bellman equation studied in optimal control?
15. Can MLT-3 be represented using a finite-state dynamical system?
16. What changes in the trellis when the number of states increases?
17. How does encoder memory impose constraints on the set of possible waveforms?
18. Can line coding therefore be viewed as designing a constrained trajectory through signal space?

28 Looking Ahead

The line coding experiment can now be viewed from several complementary perspectives.

At the circuit level,

logic gates + flip-flops

implement the encoder.

At the dynamical-systems level,

$s_{k+1} = f(s_k, b_k)$

describes its evolution.

At the signal-space level,

$$x(t) = \sum_k a_k p(t - kT)$$

describes waveform synthesis.

At the receiver,

$$\langle r, p_k \rangle$$

extracts signal-space information through correlation.

For a memoryless system, this may lead directly to a symbol decision.

For a finite-state system such as AMI or MLT-3, however,

$$\text{correlation} \rightarrow \text{state estimation} \rightarrow \text{sequence detection}$$

becomes the natural viewpoint.

Finally,

$$\text{sequence detection} \rightarrow \text{Bellman recursion} \rightarrow \text{dynamic programming}$$

connects communication theory to optimal control.

The same mathematical structure therefore appears in apparently different subjects:

Line Coding	$\longleftrightarrow$	Finite-State Dynamics	$\longleftrightarrow$	Trellis Detection	$\longleftrightarrow$	Optimal Control
Waveform	$\longleftrightarrow$	State Trajectory	$\longleftrightarrow$	Optimal Path	$\longleftrightarrow$	Bellman/HJB

This is the deeper message of the experiment:

A communication system is not merely a collection of circuits for converting bits into voltages. It is a system for designing trajectories in signal space and subsequently inferring those trajectories from noisy observations.

29 Suggested Extension: Draw the AMI Trellis

As an additional exercise, construct the AMI trellis for a short sequence of four or five bit intervals.

Use the two states

$$s_k \in \{-1, +1\}$$

and label every branch with

$$b_k/a_k.$$

Then assume that the receiver produces observations

$$z_0, z_1, \dots, z_{N-1}.$$

Assign a branch metric

$$\gamma_k = |z_k - a_k|^2$$

to every branch.

Students should then:

1. calculate the path metrics,
2. eliminate inferior paths arriving at the same state,
3. identify the surviving path,
4. and recover the estimated bit sequence.

This exercise provides a concrete bridge between the physical AMI waveform observed on the oscilloscope and the abstract Bellman/Viterbi recursion.

30 Suggested Extension: Compare AMI and MLT-3

Repeat the same construction for MLT-3.

Students should identify:

1. the state set,
2. the state-transition function,
3. the output associated with each transition,
4. the resulting trellis,
5. and the corresponding sequence-detection problem.

The objective is not merely to memorize two line-coding schemes. Rather, the objective is to recognize the common architecture

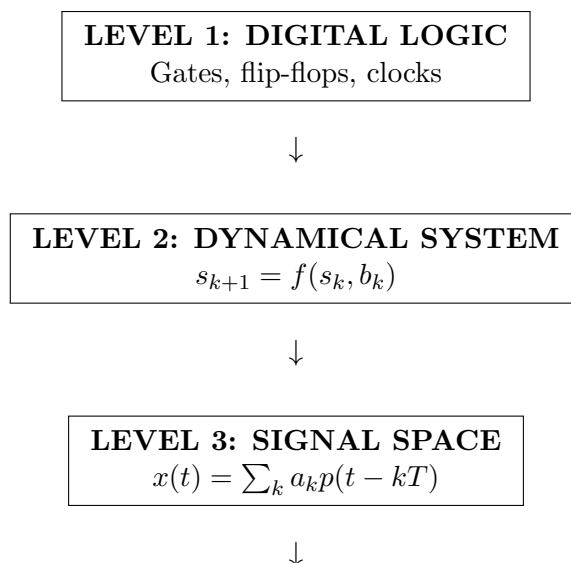
input $\rightarrow$ finite-state dynamics $\rightarrow$ waveform

and

received waveform $\rightarrow$ statistics $\rightarrow$ trellis $\rightarrow$ optimal path.

31 Final Conceptual Summary

The experiment may ultimately be summarized by the following hierarchy:



LEVEL 4: CORRELATION

$\langle r, p_k \rangle$



LEVEL 5: TRELLIS

valid state trajectories



LEVEL 6: DYNAMIC PROGRAMMING

Bellman / Viterbi recursion



LEVEL 7: CONTROL-THEORETIC VIEW

value functions / HJB analogy

Thus, the line-coding laboratory becomes an opportunity to see one object from many mathematical viewpoints:

circuit = dynamical system = signal generator = finite-state machine = trellis = dynamic-programming problem